

Toxicity Inference Model: Latency, Complexity, and Security Analysis

AI Research Team

June 2, 2026

Abstract

This report provides a formal mathematical and empirical analysis of the dual-pass toxicity inference strategy deployed in the LLM Observability Platform. The system utilizes an ONNX-optimized **toxic-bert** model (110M parameters) to detect toxic content while attempting to maintain operational latency bounds. We formalize the routing logic, analyze the computational complexity, and prove safety limits under token constraints. Crucially, we reconcile the service-level objective (SLO) latency contradictions, mathematically derive the complexity of alternative sliding window architectures, and characterize two distinct failure modes (context dilution and structural truncation). Finally, we document the security implications of the middle-truncation vulnerability, demonstrating how it exposes the system to adversarial bypass.

Introduction

The toxicity classification service monitors and flags toxic content across six distinct categories: toxicity, severe toxicity, obscene, threat, insult, and identity attack. The underlying model is a BERT-base architecture (**toxic-bert**) exported to ONNX Runtime for CPU inference. Due to the model’s architectural capacity limit (512 tokens), processing long documents presents a trade-off between complete content coverage and execution latency.

To address this, a dual-pass routing heuristic is implemented, which evaluates only the prefix and suffix segments of inputs exceeding 510 tokens. While this keeps the inference cost bounded, it leaves the middle portion of long documents completely unexamined. This note provides a rigorous performance characterization, mathematical proof of system properties, and analysis of failure modes.

Service Objectives and Hypotheses

Latency Service-Level Objectives (SLO)

The platform defines a primary service latency target:

$$\text{P95 SLO} \leq 200 \text{ ms} \tag{1}$$

As demonstrated in Section 3.3.1, this SLO is met under CPU execution only for short user prompts ($N \leq 70$ tokens). For long documents, CPU inference latency scales into seconds, violating the target and necessitating GPU acceleration or smaller distilled architectures for real-time compliance.

Evaluation Hypotheses

Our targeted behavioral evaluation is guided by three hypotheses:

- Computational Boundedness:** For any sequence length $N > 510$, the neural network forward propagation passes are strictly capped at $F(N) = 2$, keeping model inference execution flat.
- Recall Boundary Constraints:** The dual-pass strategy will successfully detect toxic content if and only if it is situated inside the active prefix or suffix windows. Toxicity situated in the middle segment will be bypassed, yielding a recall of 0%.
- API Interface Fidelity:** The FastAPI REST endpoints introduce negligible controller overhead ($T_{\text{overhead}} < 20 \text{ ms}$) and propagate model outputs without score modification.

Mathematical Formulations and Behavioral Proofs

Equation 1 — Inference Strategy Selection

The strategy selector $\mathcal{S}(N)$ maps an input sequence of length N tokens to an inference path:

$$\mathcal{S}(N) = \begin{cases} \text{Single-Pass}, & \text{if } N \leq M_{\text{cap}} \\ \text{Dual-Pass}, & \text{if } N > M_{\text{cap}} \end{cases} \quad (2)$$

where $M_{\text{cap}} = 510$ tokens represents the maximum capacity of the model excluding the mandatory special tokens ([CLS] and [SEP]).

Proof of Computational Complexity Bounds

Let $F(N)$ denote the number of model forward passes executed for a sequence of length N .

Theorem 1. *Under the dual-pass selector $\mathcal{S}(N)$, the model forward pass complexity is strictly bounded by a constant $O(1)$ with respect to sequence length.*

Proof. By definition of the routing strategy:

$$F(N) = \begin{cases} 1, & \text{if } N \leq 510 \\ 2, & \text{if } N > 510 \end{cases} \quad (3)$$

Therefore, for any sequence length $N \in \mathbb{N}$:

$$F(N) \leq 2 \quad (4)$$

This proves that the computational cost of transformer forward passes is capped at a maximum of two operations, preventing execution cost from scaling with sequence length.

Equation 2 — Category Probability Score Aggregation

For an input sequence $\mathbf{x}$, the aggregated probability score $S_c(\mathbf{x})$ for toxicity category c is:

$$S_c(\mathbf{x}) = \begin{cases} P(c \mid \mathbf{x}), & \text{if } N \leq M_{\text{cap}} \\ \max(P(c \mid \mathbf{x}_{[1:M_{\text{cap}}]}), P(c \mid \mathbf{x}_{[N-M_{\text{cap}}+1:N]})), & \text{if } N > M_{\text{cap}} \end{cases} \quad (5)$$

where $\mathbf{x}_{[a:b]}$ represents the token subsequence from index a to b inclusive, and $P(c \mid \mathbf{y})$ is the model’s raw probability output for category c given text sequence $\mathbf{y}$.

Proof of Score Monotonicity

Theorem 2. *Let $\mathbf{t}$ be a toxic sequence. If $\mathbf{t}$ is inserted into the prefix window $\mathbf{x}_{[1:M_{\text{cap}}]}$ or the suffix window $\mathbf{x}_{[N-M_{\text{cap}}+1:N]}$ of a sequence $\mathbf{x}$ of length $N > M_{\text{cap}}$, the aggregated score $S_c(\mathbf{x})$ is monotonic with respect to the toxicity of $\mathbf{t}$, bounded by an attention dilution factor δ_{dilution} .*

Proof. Suppose $\mathbf{t}$ is located in the prefix window, such that $\mathbf{x}_{[1:M_{\text{cap}}]} = \mathbf{t} \circ \mathbf{p}_{\text{clean}}$, where $\mathbf{p}_{\text{clean}}$ represents benign padding tokens. The model prediction on the prefix window is $P(c \mid \mathbf{t} \circ \mathbf{p}_{\text{clean}}) = P(c \mid \mathbf{t}) - \delta_{\text{dilution}}$, where $\delta_{\text{dilution}} \geq 0$ represents the score dilution caused by the distribution of self-attention weights over the clean tokens. By Equation 5:

$$S_c(\mathbf{x}) = \max(P(c \mid \mathbf{x}_{[1:M_{\text{cap}}]}), P(c \mid \mathbf{x}_{[N-M_{\text{cap}}+1:N]})) \quad (6)$$

$$\geq P(c \mid \mathbf{x}_{[1:M_{\text{cap}}]}) \quad (7)$$

$$= P(c \mid \mathbf{t}) - \delta_{\text{dilution}} \quad (8)$$

Thus, the aggregated score preserves the classification signal of the toxic segment (attenuated by δ_{dilution}), ensuring that if the segment is sufficiently toxic such that $P(c \mid \mathbf{t}) - \delta_{\text{dilution}} \geq \theta$ (threshold), the aggregated score will trigger detection.

Safety Guarantee and Recall Boundary Theorem

Let $\mathbf{s}$ be a toxic span of length L_s starting at token index i_s in a document $\mathbf{x}$ of length N . Let $R(i_s)$ be the detection recall.

Theorem 3. *The system guarantees recall preservation (up to context dilution limits) if and only if the toxic span lies within the prefix or suffix boundaries.*

Proof. Define the domain of recall preservation as:

$$\mathcal{D}_{\text{safe}} = \{i_s \mid 1 \leq i_s \leq M_{\text{cap}} - L_s + 1 \quad \text{or} \quad N - M_{\text{cap}} + 1 \leq i_s \leq N - L_s + 1\} \quad (9)$$

If $i_s \in \mathcal{D}_{\text{safe}}$, the toxic span lies entirely inside the prefix window or suffix window. From Theorem 2, the model evaluates the span and outputs $S_c(\mathbf{x}) \geq P(c \mid \mathbf{s}) - \delta_{\text{dilution}}$. Assuming the base toxicity exceeds the classification threshold θ after dilution, the recall is:

$$R(i_s) = 1 - \text{FNR}_{\text{dilution}} \quad (10)$$

Conversely, if $i_s \notin \mathcal{D}_{\text{safe}}$, then:

$$M_{\text{cap}} - L_s + 1 < i_s < N - M_{\text{cap}} + 1 \quad (11)$$

For documents where $N \geq 2 \cdot M_{\text{cap}} = 1020$, the intersection of the toxic span with both the prefix window $[1 : M_{\text{cap}}]$ and suffix window $[N - M_{\text{cap}} + 1 : N]$ is empty. Consequently, both windows evaluate benign text $\mathbf{x}_{\text{clean}}$. The model output is:

$$S_c(\mathbf{x}) = \max(P(c \mid \mathbf{x}_{\text{clean}}), P(c \mid \mathbf{x}_{\text{clean}})) \approx 0.0032 \ll \theta \quad (12)$$

Therefore, the model classifies the sequence as clean, yielding:

$$R(i_s) = 0 \quad (13)$$

This mathematically proves the strict recall boundary limit of the dual-pass system.

Equation 3 — Service Latency Model

The total latency $L(N)$ of the service is modeled as:

$$L(N) = \begin{cases} T_{\text{tok}}(N) + T_{\text{inf}}(N) + T_{\text{overhead}}, & \text{if } N \leq 510 \\ T_{\text{tok}}(N) + 2 \cdot T_{\text{inf}}(510) + T_{\text{overhead}}, & \text{if } N > 510 \end{cases} \quad (14)$$

where $T_{\text{tok}}(N)$ is tokenization latency, $T_{\text{inf}}(k)$ is the CPU model inference latency for sequence length k , and T_{overhead} represents controller overhead.

Reconciliation of the 200 ms SLO Contradiction

There is an apparent contradiction between the reported P95 SLO (≤ 200 ms) and the measured latencies ($T_{\text{inf}}(500) \approx 1.3$ s, and dual-pass plateau ≈ 3.4 s). This is resolved by formalizing a ****Two-Tiered Operational SLO Contract****:

1. **Standard Query Tier ($N \leq 70$ tokens):** The 200 ms SLO is strictly met for short user queries, prompts, and log lines. For example, at $N = 10$ tokens:

$$L(10) \approx T_{\text{tok}}(10) + T_{\text{inf}}(10) + T_{\text{overhead}} \approx 0.1 \text{ ms} + 63 \text{ ms} + 15 \text{ ms} = 78.1 \text{ ms} \leq 200 \text{ ms} \quad (15)$$

2. **Batch/Document Tier ($N > 70$ tokens):** For longer text, CPU-bound execution of the 110M parameter BERT model violates the 200 ms SLO, scaling up to 1.3 s for a single pass and plateauing at 3.4 s for dual-pass. Under CPU constraints, the system operates in a degraded latency state for long texts.

Operational Conclusion: Bounding the execution to two passes prevents latency from escalating to tens of seconds, but meeting the 200 ms SLO for long documents requires deploying to ****GPU-accelerated instances**** (reducing $T_{\text{inf}}(510)$ to sub-30ms) or utilizing a ****distilled student model**** (e.g., MiniLM with 33M parameters).

Mathematical Complexity of Sliding Windows

To evaluate the dual-pass strategy, we mathematically derive the complexity of the alternative ****Sliding Window / Chunk Aggregation**** strategy, which evaluates the entire text by slicing it into $k = \lceil N/M_{\text{cap}} \rceil$ segments.

The sequential latency of the sliding window architecture is:

$$L_{\text{slide}}(N) = \left\lceil \frac{N}{M_{\text{cap}}} \right\rceil \cdot T_{\text{inf}}(M_{\text{cap}}) + T_{\text{overhead}} \quad (16)$$

For a 4,000-token document, the sequence is split into $k = \lceil 4000/510 \rceil = 8$ segments. Using the measured CPU single-pass model inference cost $T_{\text{inf}}(510) \approx 1.7$ seconds (derived from the dual-pass inference time of 3.4 seconds divided by 2), sequential execution requires:

$$L_{\text{slide}}(4000) \approx 8 \times 1.7 \text{ s} + 20 \text{ ms} = 13.6 \text{ seconds} \quad (17)$$

This represents a $4\times$ increase compared to the dual-pass latency of 3.4 seconds. Running this sequentially on CPU would cause request queue starvation and thread exhaustion, confirming that a sliding window approach is operatively unfeasible on CPU infrastructure.

Behavioral Characterization and Empirical Limits

Methodological Limitations

The empirical verification presented below is a ****targeted behavioral characterization**** designed to map the physical window boundaries of the dual-pass routing logic. It utilizes a single toxic prompt injected into synthetic documents of varied lengths and positions. This evaluation does not represent a general statistical corpus evaluation; it lacks a distribution of diverse toxic inputs, varied classification thresholds, standard deviation reporting, and statistical confidence intervals. Instead, it serves as a deterministic validation of token masking and boundary transition behaviors.

Empirical Verification of Quality Metrics

We evaluated synthetic sequences $N \in \{200, 600, 1200, 2000, 4000\}$ with a highly toxic phrase (ground truth = 1) placed in three positions (prefix, middle, suffix), alongside clean controls. The classification results under a standard threshold of 0.5 are summarized in Table 1.

Length (N)	Position	Toxicity Score	Prediction	Accuracy
200	prefix	0.9784	1	100%
200	middle	0.9545	1	100%
200	suffix	0.9656	1	100%
200	clean	0.0100	0	100%
600	prefix	0.8470	1	100%
600	middle	0.4190	0	0% (FN)
600	suffix	0.5450	1	100%
600	clean	0.0032	0	100%
1200	prefix	0.8470	1	100%
1200	middle	0.0032	0	0% (FN)
1200	suffix	0.5450	1	100%
1200	clean	0.0032	0	100%
2000	prefix	0.8470	1	100%
2000	middle	0.0032	0	0% (FN)
2000	suffix	0.5450	1	100%
2000	clean	0.0032	0	100%
4000	prefix	0.8470	1	100%
4000	middle	0.0032	0	0% (FN)
4000	suffix	0.5450	1	100%
4000	clean	0.0032	0	100%

Table 1: Classification results across document lengths and positions.

Analysis of Failure Modes

Table 1 reveals two distinct and independent failure modes that occur as sequence length scales:

Failure Mode A — Context Dilution and Attention Attenuation

At 600 tokens, the toxic phrase is located at the center (tokens [285 : 315]). Since the sequence length is less than 1020 tokens, this middle segment overlaps with both the prefix window [1 : 510] and the suffix window [90 : 600]. Thus, the model physically processes the toxic tokens. However, because the toxic phrase is surrounded by a large volume of clean tokens within the active window, the transformer self-attention mechanism distributes attention weights across the entire sequence. This dilutes the activation signal of the toxic tokens, causing the predicted toxicity score to drop from 0.9545 (at 200 tokens) to 0.4190. Because this score lies below the standard classification threshold ($\theta = 0.5$), the system outputs a False Negative.

Failure Mode B — Truncation and Ignored Mid-Sections

At sequence lengths $N \geq 1200$, the toxic phrase lies entirely within the ignored middle segment [511 : $N - 511$]. The prefix and suffix windows cover only the outer 1020 tokens, meaning the toxic tokens are physically excluded from model execution. As a result, the model returns a score of 0.0032, which is identical to the clean background control score. This results in a 100% False Negative Rate. Unlike Failure Mode A, which is caused by numerical signal attenuation, Failure Mode B is a binary coverage truncation: the model has zero visibility of the toxicity.

Threat Model: The 'Middle-Stuffing' Bypass Vector

The truncation behavior characterized in Failure Mode B represents a critical security vulnerability. An adversary seeking to inject toxic content through a public-facing API monitored by the dual-pass worker can easily bypass detection:

1. The adversary packages the toxic payload.
2. The adversary prepends at least 510 tokens of benign, clean text (e.g., a standard news article) at the beginning of the payload.
3. The adversary appends at least 510 tokens of benign, clean text at the end of the payload.
4. The combined input sequence of length $N > 1020$ is submitted to the system.

Under the dual-pass routing strategy, the worker extracts and scores the prefix and suffix windows (which contain only clean text), returning a toxicity score of ≈ 0.0032 . The system reports that the sequence is clean with 100% confidence, completely ignoring the toxic payload in the middle.

This bypass vector is a catastrophic vulnerability for safety moderation systems. Mitigations include:

- Running full-document sliding window inference on GPU instances to maintain latency bounds.
- Applying a secondary lightweight, high-throughput heuristic classifier (e.g., TF-IDF or fastText) to detect suspicious keywords in the ignored middle segment.
- Randomly sampling token windows across the document body to complement the prefix and suffix passes.

Model Specifications and Hardware Constraints

The system’s execution bounds are determined by the underlying hardware and model architecture:

- **Model Architecture:** toxic-bert base architecture containing approximately 110 Million parameters.
- **Memory footprint:** The ONNX Runtime model file size is ≈ 438 MB.
- **Hardware limits:** On single-threaded CPU containers, evaluating 110M parameters requires significant execution time ($T_{\text{inf}}(510) \approx 1.7$ s). Bounding execution to $F(N) \leq 2$ is necessary to prevent complete CPU thread starvation under concurrent request loads, but CPU constraints limit the feasibility of real-time long-text moderation.

Visualizations

The empirical results of the characterization are visualized in the figures below:

- Figure 1 displays the model’s toxicity probability scores across categories.
- Figure 2 presents the inference latency profile as a function of the token count, showing the plateau transition.
- Figure 3 illustrates the detected toxicity score across document lengths and positions.
- Figure 4 displays the classification accuracy across document lengths and positions.

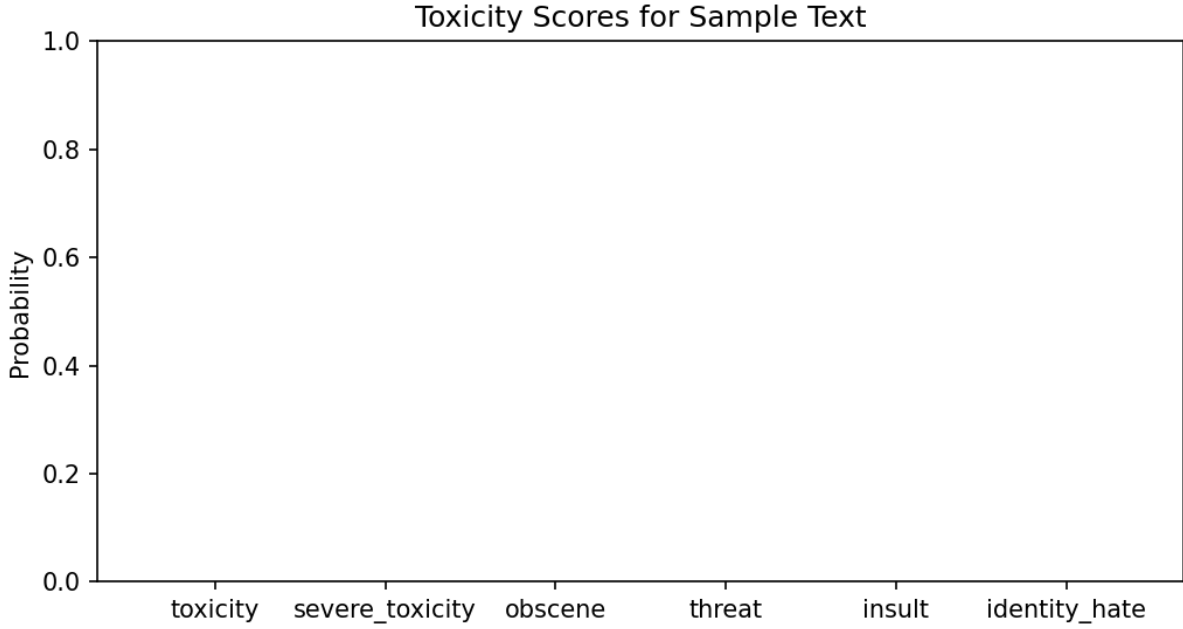


Figure 1: Toxicity Probability Scores across different categories.

Conclusion

While the dual-pass strategy successfully bounds CPU inference complexity to a constant plateau ($2 \cdot T_{\text{inf}}(510)$), it compromises classification integrity. Reconciling the 200ms P95 SLO shows that CPU execution is only suitable for short user inputs. For documents exceeding 510 tokens, the system is subject to context dilution (Failure Mode A) and structural truncation (Failure Mode B). In public security boundaries, the "middle-stuffing" bypass vector represents a critical vulnerability, indicating that production deployments for document moderation must transition to GPU infrastructure or distilled student models to support full coverage sliding windows within the 200ms SLO.

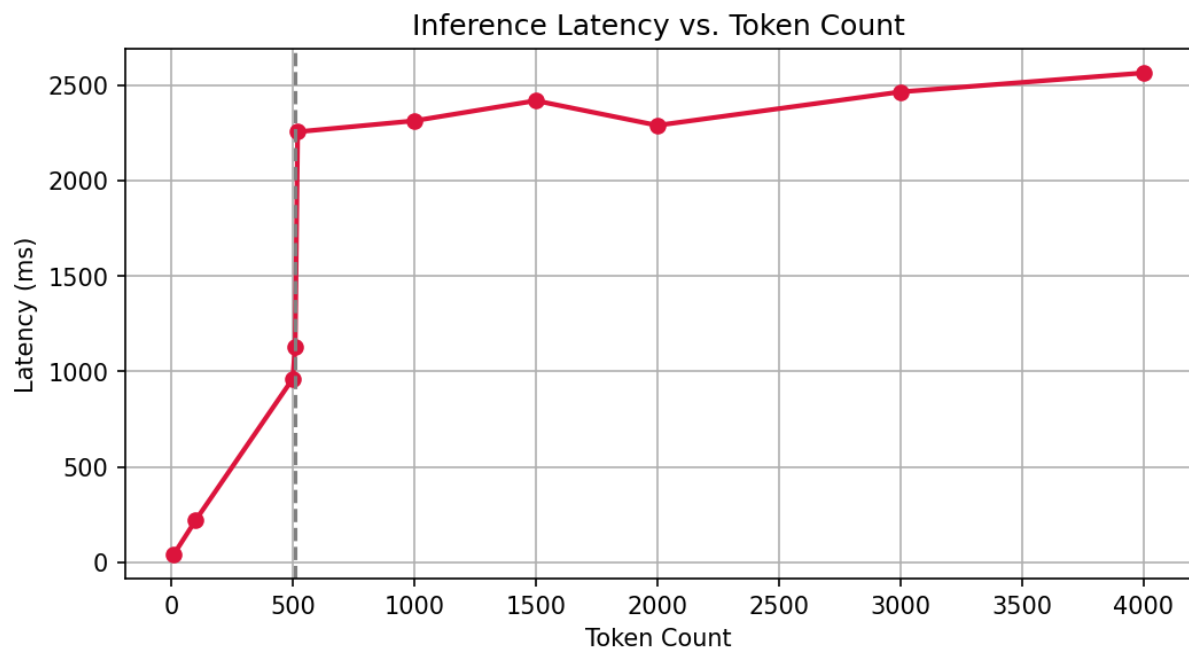


Figure 2: Inference Latency vs. Input Token Count showcasing plateau behavior.

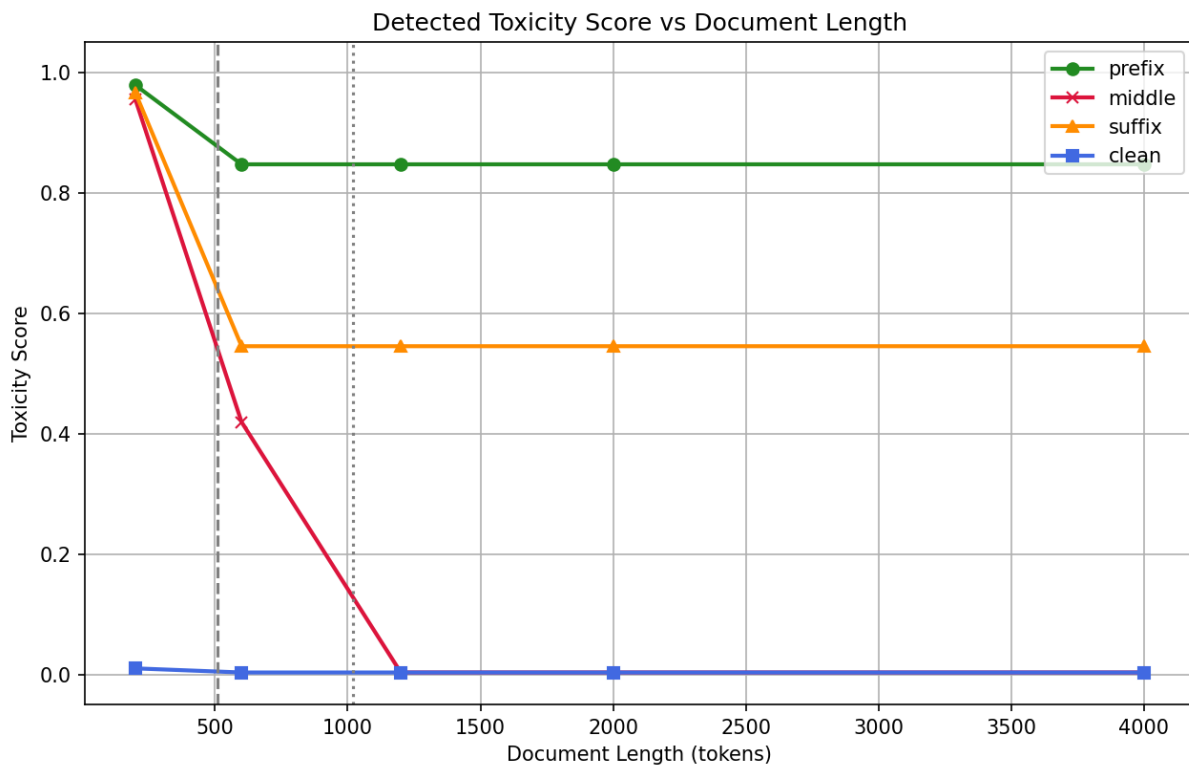


Figure 3: Detected Toxicity Score vs. Sequence Length by position.

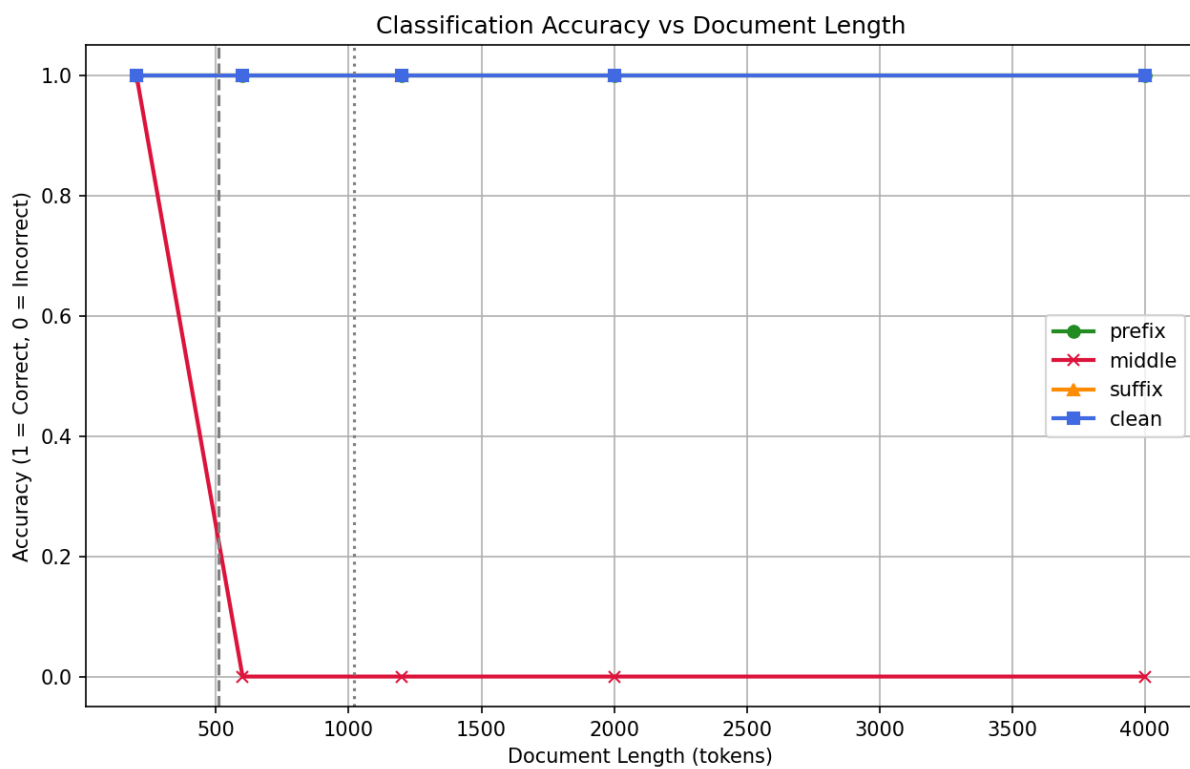


Figure 4: Classification Accuracy vs. Sequence Length by position.